The Affine One-Wayness (AOW): A Transparent Post-Quantum Temporal Verification via Polynomial Iteration

MINKA MI NGUIDJOI Thierry Emmanuel[a,*]

[a]*Laboratory of Mathematical Engineering and Information Systems (LIMSI), National Advanced School of Engineering, University of Yaoundé I, Cameroon*

Abstract

Distributed systems require robust, transparent mechanisms for verifiable temporal ordering to operate without trusted authorities or synchronized clocks. This paper introduces *Affine One-Wayness (AOW)*, a new cryptographic primitive for post-quantum temporal verification based on iterative polynomial evaluation over finite fields. AOW provides strong temporal binding guarantees by reducing its security with a *tight reduction* to the hardness of the discrete logarithm problem in high-genus hyperelliptic curves (HCDLP) and with a reduction to the Affine Iterated Inversion Problem (AIIP), which possesses dual foundations in multivariate quadratic algebra and the arithmetic of high-genus hyperelliptic curves. We present a construction with transparent setup and prove formal security against both classical and quantum adversaries. Furthermore, we demonstrate efficient integration with STARK proof systems for zero-knowledge verification of sequential computation with logarithmic scaling. As the core reliability component of the Chaotic Affine Secure Hash (CASH) framework, AOW enables practical applications in Byzantine-resistant event ordering and distributed synchronization with provable security guarantees under standard cryptographic assumptions.

Keywords: Affine One-Wayness (AOW), Post-Quantum Cryptography, Verifiable Delay Functions (VDFs), Affine Iterated Inversion Problem (AIIP), Temporal Binding, Zero-Knowledge Proofs, STARKs, Byzantine Fault Tolerance, Distributed Synchronization

1. Introduction

Distributed systems require verifiable temporal ordering without synchronized clocks or trusted authorities. While Verifiable Delay Functions (VDFs) [5, 11, 10] provide sequential computation guarantees, existing constructions face limitations: RSA-based VDFs require trusted setup, while Wesolowski's construction lacks transparent post-quantum security. This work introduces *Affine One-Wayness* (AOW), a temporal verification primitive based on polynomial iteration over finite fields that achieves transparent setup with formal post-quantum security guarantees rooted in the Affine Iterated Inversion Problem (AIIP) [22].

1.1. The Temporal Verification Challenge

Consider n participants maintaining local states $\{s_i\}_{i=1}^{n}$ who must establish verifiable event ordering $e_1 \prec e_2 \prec \cdots \prec e_k$ without global synchronization. The security requirement is *temporal binding*: no adversary can produce convincing evidence of alternative ordering without performing the requisite sequential computation. Existing solutions either rely on trusted setup (RSA-based VDFs) or lack rigorous post-quantum security analysis.

*Corresponding author

Email address: minkathierry@gmail.com (MINKA MI NGUIDJOI Thierry Emmanuel)

1.2. Technical Overview

For polynomial $f \in \mathbb{F}_q[x]$ of degree $d \geq 2$ (Definition 2.2), the AOW function computes:

$$\mathsf{AOW}_f^{(n)}(x) = f^{(n)}(x) \tag{1}$$

where $f^{(n)}$ denotes n-fold composition (Definition 2.1). The security reduces directly to AIIP: given $y = f^{(n)}(x)$, finding any preimage x' such that $f^{(n)}(x') = y$ is computationally infeasible under Assumption 2.1.

Theorem 1.1 (Temporal Binding of AOW). *Under Assumption 2.1, AOW provides temporal binding (Definition 2.4) with security parameter λ: for any probabilistic polynomial-time adversary $\mathcal{A}$,*

$$\Pr[\mathcal{A} \textit{ wins temporal forgery game}] \leq \textit{negl}(\lambda) \tag{2}$$

with parameters chosen according to Table 1.

1.3. The CASH Framework

The Chaotic Affine Secure Hash **(CASH)** framework is introduced as a primitive-level instantiation of the Quantum Composable and Contextual Security Infrastructure **(Q2CSI)** [25]. It is defined as the triplet

$$\text{CASH} = (\text{CEE}, \text{AOW}, \text{SH}), \tag{3}$$

whose name (CASH) is drawn from the composing primitives themselves: **C** from CEE, **A** from AOW, and **SH** from SH, highlighting how each component realizes one property of the Confidentiality, Reliability, Opposability **(CRO)** trilemma [24]:

- **CEE** (Chaotic Entropic Expansion) ensures *Confidentiality* through entropy-preserving expansion.
- **AOW** (Affine One-Wayness) ensures *Reliability* through temporal binding and verifiable ordering.
- **SH** (Semantic Holder) ensures *Opposability* through legally interpretable semantic anchoring.

The isolation and formal specification of these three properties, Confidentiality, Reliability, and Opposability (CRO), enable the construction of systems with end-to-end verifiable security guarantees.

Remark 1.1 (Framework Roadmap). *AOW constitutes the second primitive in the CASH framework, following the Chaotic Entropic Expansion (CEE) primitive for confidentiality [23]. The companion primitive SH (Semantic Holder) for legal opposability will be formally defined in subsequent work. This modular approach allows independent analysis of each CRO property while maintaining composability within the Q2CSI infrastructure. With AOW, the CASH framework now provides both confidentiality (via CEE) and reliability (via AOW), establishing two of the three pillars required for complete CRO coverage.*

1.4. Our Contribution: AOW

We construct AOW as the reliability component of the Chaotic Affine Secure Hash (CASH) framework, complementing the Chaotic Entropic Expansion (CEE) [23] primitive for confidentiality. AOW leverages the computational hardness of the AIIP to provide:

1. **Transparent Temporal Anchoring**: No trusted setup or hidden parameters required.
2. **Provable Post-Quantum Security**: Reduction to AIIP with quantum resistance based on high-genus hyperelliptic curve DLP (Theorem 2.1) and MQ hardness (Theorem 2.2).
3. **Efficient Zero-Knowledge Verification**: Native integration with STARK proof systems via algebraic intermediate representation (AIR) with $O(\lambda \log n)$ verification complexity (Theorem 5.1).

This work establishes the theoretical and practical foundation for the reliability pillar (AOW) of the CASH framework, enabling the future construction of the SH primitive for full-stack legal opposability.

1.5. Paper Organization

The remainder of this paper is organized as follows. Section 2 establishes the notation, formally defines the Affine Iterated Inversion Problem (AIIP), and reviews the required security definitions for temporal binding. Section 3 presents the formal construction of the AOW primitive and proves its core temporal binding properties. Section 4 provides a comprehensive security analysis, including classical and quantum resistance arguments and parameter selection guidelines. Section 5 details the integration of AOW with STARK proof systems for efficient and zero-knowledge verification. Section 6 demonstrates practical applications in Byzantine-resistant event ordering and distributed synchronization. Section 7 discusses related work and provides a comparative analysis. Finally, Section 8 summarizes our contributions and outlines directions for future work. Detailed proofs and implementation notes are provided in the appendices.

2. Preliminaries

2.1. Notation and Finite Fields

We adopt notation from [22]. Let $\mathbb{F}_q$ denote the finite field with $q = p^k$ elements, where p is an odd prime. For polynomial $f \in \mathbb{F}_q[x]$, the n-fold composition is defined recursively:

$$f^{(0)}(x) = x, \quad f^{(n)}(x) = f(f^{(n-1)}(x)) \text{ for } n \geq 1 \tag{4}$$

We denote by $\mathcal{F}_d$ the family of degree-d polynomials satisfying the conditions from Definition 2.2.

2.2. The Affine Iterated Inversion Problem (AIIP)

This work's security relies on the hardness of the *AIIP Problem*. We restate its core definition and foundational results here for self-containment.

Definition 2.1 (Iterated Composition). *For a function $f : \mathbb{F}_q \to \mathbb{F}_q$ and positive integer n, the n-fold composition of f is denoted $f^{(n)}$ and defined recursively:*

$$f^{(0)}(x) = x, \quad f^{(n)}(x) = f(f^{(n-1)}(x)) \text{ for } n \geq 1. \tag{5}$$

Definition 2.2 (AIIP). *Let $f \in \mathbb{F}_q[x]$ be a polynomial of degree $d \geq 2$. The* Affine Iterated Inversion Problem *with parameters (f, n, y) asks:*

Given f, positive integer n, and target $y \in \mathbb{F}_q$,
find $x \in \mathbb{F}_q$ such that $f^{(n)}(x) = y$.

The term *affine* refers to the most studied case where $f(x) = x^d + \alpha$, though the problem extends to general polynomials. The computational hardness of AIIP is established through two independent, complementary frameworks:

Theorem 2.1 (Connection between AIIP and HCDLP). *Let $f(x) = x^2 + \alpha$ where $\alpha \in \mathbb{F}_q^*$ is a quadratic non-residue. The problem of solving AIIP for (f, n, y) can be embedded into the problem of computing a discrete logarithm in the Jacobian of a hyperelliptic curve of genus $g_n = 2^{n-1} - 1$ over $\mathbb{F}_q$. This embedding demonstrates that any algorithm solving DLP in such high-genus Jacobians would also solve AIIP.*

Theorem 2.2 (Reduction from AIIP to MQ). *Let $\mathcal{F}_d$ be a family of polynomials over $\mathbb{F}_q$ of degree $d \geq 2$. The Affine Iterated Inversion Problem (AIIP) for any $f \in \mathcal{F}_d$ is polynomial-time many-one reducible to the problem of solving a system of multivariate quadratic equations over $\mathbb{F}_p$ ($MQ_{\mathbb{F}_p}$), where $q = p^k$.*

Furthermore, under a plausible one-wayness assumption for the iterated map, the average-case hardness of the resulting structured MQ systems is proven. See to [22] for the detailed constructions and proofs of these reductions.

Conjecture 2.1 (AIIP Hardness). *For $f(x) = x^2 + \alpha$ where $\alpha \in \mathbb{F}_q^*$ is a quadratic non-residue, and parameters satisfying $2 \leq n \leq \log_2 q$, no polynomial-time algorithm solves AIIP with probability better than $d^n/q + negl(\log q)$.*

This conjecture is supported by the reductions to the well-studied hard problems of high-genus HCDLP and MQ.

Definition 2.3 (The Polynomial Class $\mathcal{F}_d$). *Following [22], a polynomial $f \in \mathbb{F}_q[x]$ belongs to the class $\mathcal{F}_d$ if:*

1. $\deg(f) = d$ *(exact degree).*
2. *f is not exceptional (not of form $g(x)^p - g(x) + c$ for any $g \in \mathbb{F}_q[x], c \in \mathbb{F}_q$).*
3. *The critical points of f have distinct forward orbits.*

For cryptographic applications, we primarily use $\mathcal{F}_2$, the class of quadratic polynomials satisfying these conditions, typically of the form $f(x) = x^2 + \alpha$ with $\alpha \in \mathbb{F}_q^*$.

2.3. Complexity Assumptions

The security of the AOW primitive is based on the following computational hardness assumptions, justified by the results in [22].

Assumption 2.1 (Hardness of AIIP). *For a polynomial $f \in \mathcal{F}_2$ (e.g., $f(x) = x^2 + \alpha$), a uniformly chosen iteration depth $n = (\lambda)$, and a uniformly random target $y \in \mathbb{F}_q$, no probabilistic polynomial-time algorithm can find a preimage x such that $f^{(n)}(x) = y$ with non-negligible probability in the security parameter λ.*

Assumption 2.2 (Hardness of DLP in High-Genus Jacobians). *For hyperelliptic curves of genus $g = \omega(\log q)$ over $\mathbb{F}_q$, no polynomial-time algorithm solves the discrete logarithm problem in $\mathrm{Jac}(C)$ with non-negligible probability.*

Assumption 2.3 (Average-Case Hardness of MQ). *For random systems of $m = n + o(n)$ quadratic equations in n variables over $\mathbb{F}_q$, no polynomial-time algorithm finds a solution with non-negligible probability when $q = \mathrm{poly}(n)$.*

2.4. Temporal Verification Requirements

Definition 2.4 (Temporal Binding). *A function $T : \mathcal{X} \times \mathbb{N} \to \mathcal{Y}$ provides $(\lambda, n_{\max})$-temporal binding if for all $n \leq n_{\max}$:*

1. ***Sequential Evaluation**: Computing $T(x, n)$ requires $\Omega(n)$ sequential steps with probability $1 - negl(\lambda)$.*
2. ***Depth Uniqueness**: For any $n \neq m \leq n_{\max}$, $\Pr_{x \leftarrow \mathcal{X}}[T(x, n) = T(x, m)] \leq negl(\lambda)$.*
3. ***Inversion Hardness**: For any PPT adversary $\mathcal{A}$,*

$$\Pr_{\substack{x \leftarrow \mathcal{X} \\ y = T(x,n)}} [\mathcal{A}(n, y) \text{ outputs } x' \text{ with } T(x', n) = y] \leq negl(\lambda) \tag{6}$$

Definition 2.5 (Transparent Setup). *A cryptographic protocol has* transparent setup *if all public parameters can be generated deterministically from public randomness without any secret information. Specifically:*

1. *No trusted party holds secret trapdoors.*
2. *Parameters are publicly verifiable.*
3. *Setup can be reproduced by any party.*

Definition 2.6 (Temporal Anchoring). *A function $T : \mathcal{X} \times \mathbb{N} \to \mathcal{Y}$ provides* temporal anchoring *if it cryptographically binds an input to a verifiable time delay. Formally, for input x and delay parameter n:*

1. *Computing $T(x, n)$ requires $\Omega(n)$ sequential operations.*
2. *The output $y = T(x, n)$ serves as a commitment to both x and the elapsed time n.*
3. *No adversary can produce y without performing the requisite computation.*

3. The AOW Construction

3.1. Definition

Definition 3.1 (Affine One-Way Function). *The term **Affine** in AOW denotes its foundational connection to the Affine Iterated Inversion Problem (AIIP), from which it derives its security, rather than the algebraic structure of the function itself. For security parameter λ, let $\Theta = (\mathbb{F}_q, f, n_{\max})$ where:*

- *$\mathbb{F}_q$ is a finite field with $q = p^k$ and $\log_2 q \geq 2\lambda$.*
- *$f \in \mathcal{F}_2$ is a quadratic polynomial (typically $f(x) = x^2 + \alpha$ with $\alpha \in \mathbb{F}_q^*$).*

- $n_{\max} = \lfloor \min(2^{\lambda/2}, q^{1/4}) \rfloor$.

The AOW function is defined as:

$$\mathsf{AOW}_f^{(n)} : \mathbb{F}_q \to \mathbb{F}_q, \quad \mathsf{AOW}_f^{(n)}(x) = f^{(n)}(x) \tag{7}$$

for iteration depths $1 \leq n \leq n_{\max}$.

The value $n_{\max}$ is chosen such that for $q = 2^{2\lambda}$, we have $n_{\max} = 2^{\lambda/2}$, which ensures that the advantage loss in the security reduction is negligible (since $n_{\max} \cdot \epsilon \leq 2^{\lambda/2} \cdot \mathrm{negl}(\lambda) \leq \mathrm{negl}(\lambda)$) and that the depth uniqueness property holds (as shown in Theorem 3.1).

3.2. Temporal Binding Properties

Theorem 3.1 (AOW Temporal Binding). *Under Assumption 2.1, the AOW function satisfies Definition 2.4 with parameters* $(\lambda, n_{\max})$.

Proof 3.1 (Proof Sketch). *We verify the three temporal binding properties:*

1. ***Sequential Evaluation:*** *Each iteration* $x_{i+1} = f(x_i)$ *depends on* x_i*, creating inherent sequential dependency with time complexity* $\Theta(n \cdot d)$.
2. ***Depth Uniqueness:*** *For* $n \neq m$*, collision probability is bounded by the cycle structure:* $\Pr[f^{(n)}(x) = f^{(m)}(x)] \leq \frac{(m-n) \cdot 2^{m-n}}{q} \leq 2^{-\lambda}$ *for our parameters.*
3. ***Inversion Hardness:*** *Follows directly from AIIP hardness (Assumption 2.1).*

The complete proof is in Appendix B.15.

3.3. Security Model and Reduction

Definition 3.2 (Temporal Forgery Game). *The temporal forgery game* $\mathsf{Game}_{\mathcal{A}}^{\mathsf{forge}}(\lambda)$ *for an adversary* $\mathcal{A}$ *proceeds:*

1. *The challenger selects* $f \leftarrow \mathcal{F}_2$ *and sends* f *to* $\mathcal{A}$.
2. $\mathcal{A}$ *makes adaptive queries* (x_i, n_i) *for* $1 \leq n_i \leq n_{\max}$ *and receives* $y_i = f^{(n_i)}(x_i)$.
3. $\mathcal{A}$ *outputs a forgery attempt* (x^*, n^*, y^*).
4. $\mathcal{A}$ *wins if* $f^{(n^*)}(x^*) = y^*$ *and no query* (x_i, n_i) *was made with* $f^{(n_i)}(x_i) = y^*$.

Theorem 3.2 (Unified Security Reduction for CASH Framework). *Let* ϵ_{AIIP} *be the advantage against AIIP,* ϵ_{MQ} *against MQ, and* ϵ_{HCDLP} *against HCDLP. For the CASH framework with CEE providing confidentiality and AOW providing reliability, any adversary* $\mathcal{A}$ *running in time* t *making at most* q *queries satisfies:*

$$\Pr[\mathcal{A} \text{ breaks } AOW] \leq \min\left\{n_{\max} \cdot \epsilon_{AIIP}, \sqrt{n_{\max}} \cdot \epsilon_{MQ}, \epsilon_{HCDLP}\right\} \tag{8}$$

where the $\sqrt{n_{\max}}$ *factor for MQ follows from the hybrid argument over iteration depth. The reduction to the hyperelliptic curve discrete logarithm problem (HCDLP) is* tight.

Proof 3.2 (Proof Sketch). *We provide three independent reductions:*

1. ***Direct AIIP reduction:*** *A standard guessing argument for the forgery iteration depth* n^* *yields a non-tight factor of* $n_{\max}$.

2. ***MQ reduction via unrolling:*** *A hybrid argument over $\sqrt{n_{\max}}$ intermediate checkpoints reduces the loss factor to $\sqrt{n_{\max}}$.*

3. ***HCDLP embedding:*** *A direct transformation without multiplicative loss. This reduction is* tight*: an adversary breaking AOW with advantage δ implies an algorithm solving HCDLP with advantage δ.*

The security of AOW is therefore governed by the minimum of these bounds. The tight reduction to HCDLP (Assumption 2.2) ensures that the advantage $\Pr[\mathcal{A}$ breaks $AOW]$ is negligible. The complete proof is in Appendix B.16.

Functionality 3.1 (Ideal Temporal Verification Functionality $\mathcal{F}_{\mathbf{AOW}}$). *The functionality $\mathcal{F}_{AOW}$ is parameterized by polynomial $f \in \mathcal{F}_d$ and maintains state $\mathcal{S}$.* ***Initialization:*** *Upon receiving $(\texttt{INIT}, f)$ from party P:*

- *Store f and initialize $\mathcal{S} = \emptyset$.*
- *Send $(\texttt{READY}, f)$ to all parties.*

Temporal Anchoring: *Upon receiving $(\texttt{ANCHOR}, x, n)$ from P:*

- *If $(x, n, \cdot) \in \mathcal{S}$, return $(\texttt{DUPLICATE})$ to P.*
- *Wait exactly $n \cdot \tau$ time units (enforcing sequentiality).*
- *Compute $y = f^{(n)}(x)$ internally.*
- *Store $(x, n, y, time)$ in $\mathcal{S}$.*
- *Send $(\texttt{ANCHORED}, y, n, time)$ to all parties.*

Verification: *Upon receiving $(\texttt{VERIFY}, y, n)$ from P:*

- *If $(\cdot, n, y, t) \in \mathcal{S}$, send $(\texttt{VALID}, y, n, t)$ to P.*
- *Else send $(\texttt{INVALID})$ to P.*

4. Security Analysis

4.1. Reduction to AIIP

The security of AOW is fundamentally based on the hardness of the Affine Iterated Inversion Problem (AIIP). The reduction in Theorem 3.2 shows that any adversary breaking the temporal binding of AOW can be used to solve AIIP with comparable advantage.

Tight Security via HCDLP. While the reduction to AIIP incurs a non-tightness factor of $n_{\max}$, the security of AOW does not rely on this bound alone. As shown in Theorem 3.2, the reduction to the hardness of the discrete logarithm problem in high-genus hyperelliptic curves (HCDLP, Assumption 2.2) is *tight*. This provides a strong, lossless foundation for the security of AOW. The parameter selection in Section 4.4 is based on the hardness of HCDLP, ensuring that the advantage of any adversary against AOW is bounded by ϵ_{HCDLP}, which is negligible for the chosen field size q. The non-tight reduction to AIIP is a known artifact of the proof technique for iterative primitives and does not indicate an underlying weakness in the construction. Finding a tight reduction from AIIP remains an open problem.

4.2. Classical Security Analysis

Against classical adversaries, AOW resists known attack vectors:

Proposition 4.1 (Algebraic Attack Resistance). *The most efficient method for directly solving the equation $f^{(n)}(x) = y$ is via algebraic attacks that compute a Gröbner basis of the associated system. For a generic instance, this system behaves like a semi-regular sequence [7], a property that governs the complexity of modern Gröbner basis algorithms like Faugère's F4 [19] and F5 [20]. The complexity of these algorithms is dominated by the* degree of regularity *d_{reg} of the system, requiring on the order of $\binom{n+d_{reg}}{d_{reg}}^{\omega}$ operations for linear algebra constant $\omega \approx 2.8$. Crucially, the algebraic degree of the iterated polynomial $f^{(n)}$ is $d^n = 2^n$. For $n \geq 128$, this results in a degree of regularity $d_{reg} \geq 2^{128}$, making the complexity of a Gröbner basis attack $\Omega(2^{128})$, which is computationally infeasible. This is further supported by complexity analyses of the F5 algorithm [4] for such high-degree systems.*

Proof 4.1 (Proof Sketch). *By induction: $\deg(f^{(i+1)}) = \deg(f) \cdot \deg(f^{(i)}) = 2 \cdot 2^i = 2^{i+1}$. Root finding for degree 2^{128} requires $\Omega(2^{128})$ operations. See Appendix B.9.*

Proposition 4.2 (Cycle Attack Resistance). *For a random quadratic polynomial $f \in \mathcal{F}_2$ over $\mathbb{F}_q$, the expected cycle length is $\Theta(\sqrt{q})$. With $q \geq 2^{256}$, cycle-based attacks require $\Omega(2^{128})$ operations, which is infeasible.*

Proof 4.2 (Proof Sketch). *By functional graph theory, random degree-d polynomials have expected cycle length $\sqrt{\pi q/8}$. For $q = 2^{256}$, this is $\Omega(2^{128})$. See Appendix B.10.*

Proposition 4.3 (Meet-in-the-Middle Resistance). *A meet-in-the-middle attack would require storing $O(\sqrt{q})$ intermediate values. For $q \geq 2^{256}$, this requires $O(2^{128})$ storage, which is infeasible.*

Proof 4.3 (Proof Sketch). *The birthday paradox requires $\sqrt{q}$ evaluations to find collision. Storage of 2^{128} field elements requires exabytes of memory. See Appendix B.11.*

4.3. Quantum Security Analysis

The best known quantum algorithms for high-genus HCDLP, such as those based on [8, 6] index calculus, remain subexponential but still superpolynomial. For the exponentially large genus g in our construction, this complexity becomes quasi-exponential and infeasible

Theorem 4.1 (Quantum Resistance of AOW). *Under Assumption 2.1 and the conjecture that no quantum algorithm for solving AIIP outperforms Grover's search, AOW provides λ bits of quantum security with field size $q = 2^{2\lambda}$ and iteration count $n = \lambda$.*

Proof 4.4 (Proof Sketch). *We analyze three quantum attack vectors:*

- ***Grover:*** *Requires $O(n \cdot \sqrt{q}) = O(\lambda \cdot 2^{\lambda})$ operations.*
- ***Algebraic:*** *Exponential degree 2^n prevents efficient quantum algebraic attacks.*
- ***Underlying problems:*** *HCDLP and MQ remain hard quantumly.*

Complete analysis in Appendix B.8.

Table 1: Recommended AOW parameters for different security levels[1]

Security Level (λ)	Field Size (q)	Max Depth ($n_{\max}$)	Typical Usage (n)	Polynomial
128-bit	$q = 2^{256}$	$n_{\max} = 2^{64}$	$n = 64$	$f(x) = x^2 + \alpha$
192-bit	$q = 2^{384}$	$n_{\max} = 2^{96}$	$n = 96$	$f(x) = x^2 + \alpha$
256-bit	$q = 2^{512}$	$n_{\max} = 2^{128}$	$n = 128$	$f(x) = x^2 + \alpha$

4.4. Parameter Selection

The parameters are chosen to ensure that the tight reduction to HCDLP provides λ bits of security:

Remark 4.1. *The distinction between $n_{\max}$ (maximum supported depth) and n (typical operational depth) aligns with CEE's usage where $n = \lambda/2$ for standard applications, while $n_{\max}$ provides an upper bound for security analysis.*

The parameters are chosen to ensure:

Theorem 4.2 (Parameter Selection Optimality). *For security parameter λ, the choice $n_{\max} = 2^{\lambda/2}$ and $q = 2^{2\lambda}$ is optimal, achieving:*

1. ***Classical Security:*** *Cycle attacks require $\Omega(\sqrt{q}) = \Omega(2^{\lambda})$ operations.*
2. ***Quantum Security:*** *Grover requires $\Omega(\sqrt{q/2^{n_{\max}}}) = \Omega(2^{\lambda/2})$ operations.*
3. ***Reduction Tightness:*** *Loss factor $n_{\max} \cdot \epsilon = 2^{\lambda/2} \cdot 2^{-2\lambda} = 2^{-3\lambda/2}$ remains negligible.*
4. ***Practical Feasibility:*** *$n_{\max} = 2^{64}$ iterations for $\lambda = 128$ is computationally feasible.*

Proof 4.5 (Proof Sketch). *The optimality follows from balancing four constraints:*

$$\text{Cycle resistance:} \quad n_{\max} \leq \sqrt{q} \tag{9}$$

$$\text{Quantum resistance:} \quad n_{\max} \cdot \sqrt{q} \geq 2^{2\lambda} \tag{10}$$

$$\text{Reduction quality:} \quad n_{\max} \cdot 2^{-\lambda} \leq 2^{-\lambda/2} \tag{11}$$

$$\text{Computation bound:} \quad n_{\max} \leq 2^{80} \tag{12}$$

The intersection yields $n_{\max} = 2^{\lambda/2}$. Full derivation in **??**.

5. Zero-Knowledge Temporal Proofs

5.1. AIR Encoding for AOW

To integrate with STARK proof systems, we encode AOW computation as an Algebraic Intermediate Representation (AIR):

Definition 5.1 (AOW Execution Trace). *For computing $y = f^{(n)}(x)$ with polynomial $f \in \mathcal{F}_d$ of degree d, the execution trace $T \in \mathbb{F}_q^{n \times (d+1)}$ has rows:*

$$T[i] = (x_i, c_{i,1}, c_{i,2}, \ldots, c_{i,d-1}, x_{i+1}) \quad \text{for } i = 0, \ldots, n-1 \tag{13}$$

where:

- *$x_0 = x$ (input) and $x_{i+1} = f(x_i)$ (iteration).*

- $c_{i,j} = x_i^{j+1}$ *for* $j = 1, \ldots, d-1$ *(intermediate powers).*

This representation enables efficient verification of polynomial evaluation at each step.

Definition 5.2 (AOW AIR Constraints). *The AIR constraints for the trace* T *are:*

$$\begin{aligned} \textit{Transition:} \quad & T[i,1] = T[i,0]^2 + \alpha \quad \textit{for } i = 0, \ldots, n-1 && (14)\\ \textit{Boundary:} \quad & T[0,0] = x && (15)\\ & T[n-1,1] = y && (16) \end{aligned}$$

The transition constraint ensures each step correctly applies f, and the boundary constraints fix the input and output values.

5.2. STARK Integration

Theorem 5.1 (Efficiency of AOW-STARK). *The AOW computation admits a STARK proof [14] with*

- *Proof size:* $O(\lambda \log n)$ *field elements.*
- *Verification time:* $O(\lambda \log n)$ *field operations.*
- *Prover time:* $O(n \log n)$ *field operations.*

Proof 5.1 (Proof Sketch). *The trace has* n *rows and* $d+1$ *columns for degree-*d *polynomial. Using FRI protocol with security parameter* λ *and blow-up factor* $\rho = 4$*:*

- *Merkle paths:* $O(\lambda)$ *paths of depth* $O(\log n)$*.*
- *FRI rounds:* $O(\log n)$ *with* $O(\lambda)$ *queries per round.*
- *FFT dominates prover time:* $O(n \log n)$*.*

Complete analysis in Appendix B.17.

5.3. Zero-Knowledge Property

Proposition 5.1 (Zero-Knowledge of AOW-STARK). *The STARK proof for AOW is zero-knowledge: it reveals no information about the input* x *beyond the output* y *and the fact that* $y = f^{(n)}(x)$*.*

Proof 5.2 (Proof Sketch). *The simulator generates random trace satisfying boundary constraint* $T'[n-1,1] = y$ *and uses STARK simulator. Masking polynomials ensure statistical hiding. Complete simulator construction in Appendix B.12.*

6. Applications

6.1. Byzantine Event Ordering

Definition 6.1 (Temporal Event Chain). *For events* $e_1, \ldots, e_k$*, the temporal chain is constructed as:*

$$\tau_i = \mathsf{AOW}_f^{(i)}(H(e_i \| \tau_{i-1})) \tag{17}$$

where $H : \{0,1\}^* \to \mathbb{F}_q$ *is a cryptographic hash function, and* τ_0 *is a fixed genesis value.*

Theorem 6.1 (Byzantine Resistance of Temporal Chain). *Under Assumption 2.1, no coalition of Byzantine participants can alter the event ordering without performing the requisite sequential computation.*

Proof 6.1 (Proof Sketch). *Altering the order of events would require finding a collision in the chain. Specifically, for any attempt to swap events e_i and e_j ($i < j$), the adversary would need to find τ' such that:*

$$\mathsf{AOW}_f^{(j)}(H(e_j \| \tau_{i-1})) = \mathsf{AOW}_f^{(j)}(H(e_i \| \tau')) \tag{18}$$

This reduces to solving AIIP for depth j, which is infeasible under Assumption 2.1. The sequential nature of AOW evaluation (Theorem 3.1) ensures that any attempt to recompute the chain with altered ordering requires performing all intermediate computations sequentially.

6.2. Distributed Synchronization

Definition 6.2 (Synchronization Protocol). *Participants prove completion of computational round r by publishing:*

$$proof_r = (\tau_r, \pi_r) \tag{19}$$

where $\tau_r = \mathsf{AOW}_f^{(n_r)}(state_r)$ and π_r is a STARK proof (Theorem 5.1) attesting that τ_r was correctly computed.

Proposition 6.1 (Synchronization Security). *Participants cannot claim completion of round r without actually performing n_r sequential operations.*

Proof 6.2. *The temporal binding property of AOW (Theorem 3.1) ensures that computing τ_r requires exactly n_r sequential steps. The STARK proof π_r guarantees that the computation was performed correctly (by the soundness property of STARKs) without revealing the intermediate state. Any attempt to shortcut the computation would either violate the AIIP assumption (Assumption 2.1) or the soundness of the STARK system.*

Table 2: Concrete complexity comparison for 128-bit security

Operation	AOW	RSA-VDF	Wesolowski	Sloth
Setup	$O(1)$	$O(2^{2048})$	$O(2^{2048})$	$O(1)$
Evaluation (sequential)	$64 \cdot T_{\text{mul}}$	$2^{20} \cdot T_{\text{exp}}$	$2^{20} \cdot T_{\text{exp}}$	$2^{20} \cdot T_{\text{sqrt}}$
Proof Generation	$O(64 \log 64)$	$O(1)$	$O(\log 2^{20})$	N/A
Verification	$O(128 \log 64)$	$O(1)$	$O(1)$	$O(1)$
Proof Size	≈ 10 KB	≈ 256 B	≈ 256 B	N/A

Remark 6.1. *T_{mul} denotes field multiplication in $\mathbb{F}_{2^{256}}$ (≈ 10 ns), T_{exp} denotes modular exponentiation in 2048-bit RSA group (≈ 1 ms), and T_{sqrt} denotes modular square root (≈ 0.5 ms).*

7. Discussion and Related Work

The Affine One-Wayness (AOW) primitive establishes a new paradigm for verifiable temporal sequencing by leveraging iterative polynomial maps over finite fields, intersecting with several areas of cryptography including Verifiable Delay Functions (VDFs), iterative cryptographic constructions, and post-quantum secure protocols.

7.1. Positioning Within Verifiable Delay Functions

The concept of Verifiable Delay Functions was formally introduced by Boneh et al. [5] as functions that require prescribed sequential steps to evaluate yet enable efficient verification. While AOW shares this core objective, it differs fundamentally in its underlying hardness assumption and construction technique. Existing VDF constructions primarily fall into three categories [5, 11, 10]:

1. **RSA-based VDFs** [5, 10] rely on sequential squaring in RSA groups but require trusted setup, introducing potential failure points.

2. **Algebraic VDFs** [11] use groups of unknown order but depend on number-theoretic assumptions vulnerable to quantum attacks.

3. **Isogeny-based VDFs** [9, 2] often suffer from large proof sizes or computational overhead, though recent work aims to make them more practical.

4. **Vector and isogeny-based VDFs** [18, 16] often suffer from large proof sizes or computational overhead.

AOW distinguishes itself by leveraging polynomial iteration over finite fields rather than group-based sequential squaring, enabling transparent setup and post-quantum security while maintaining efficient verification through STARK proof systems (Theorem 5.1).

7.2. Iterative Constructions and Security Foundations

The use of iterative structures in cryptography has been explored in various contexts. The MiMC family [1] utilizes iterative polynomial evaluations for symmetric cryptography, while Habutsu et al. [12] proposed chaotic maps for encryption, though lacking rigorous security analysis. More recent works like Arion [21] and Reinforced Concrete [17] demonstrate growing interest in algebraic approaches. AOW's security foundation is uniquely dual, resting on the established hardness of both the Multivariate Quadratic (MQ) problem (Theorem 2.2) and the Discrete Logarithm Problem in high-genus hyperelliptic curves (Theorem 2.1) via reduction to the AIIP Problem. This bifurcated foundation provides significant robustness, a cryptanalytic advance against one underlying problem does not necessarily compromise AOW, as an adversary must break both its combinatorial-algebraic and number-theoretic structures simultaneously.

7.3. Post-Quantum Landscape and Verification Efficiency

As quantum computing advances threaten traditional cryptographic assumptions, post-quantum secure primitives have gained importance. While the NIST standardization process [13] has focused primarily on encryption and signature schemes, AOW contributes a post-quantum temporal verification primitive based on AIIP's dual foundations. The integration with STARKs [14] enables efficient verification (Theorem 5.1), though the current security reduction introduces a non-tight factor of $n_{\max}$ in the advantage bound (Theorem 3.2). While mitigated by parameter selection (e.g., $n_{\max} = 2^{\lambda/2}$), achieving a tight reduction remains an open research problem. The practical verification efficiency, while asymptotically optimal, requires concrete analysis for real-world deployment in distributed systems and blockchain protocols.

7.4. Comparative Analysis and Applications

Table 3 provides a comparative analysis of AOW against representative VDF constructions: AOW occupies a unique position by combining transparent setup, post-quantum security, and efficient verification. Applications span randomness beacons, proof-of-sequential-work consensus mechanisms, and time-release cryptography, particularly in decentralized environments requiring quantum resistance (Section 6).

Table 3: Comparison of VDF constructions

Construction	Setup	PQ	Proof Size	Verif. Time	Verif. Ops	Assumption
RSA-based [5]	Trusted	No	$O(1)$	$O(1)$	Exponentiation	Seq. Squaring
Wesolowski [10]	Trusted	No	$O(1)$	$O(1)$	Exponentiation	Low Order
Pietrzak [11]	Trusted	No	$O(\log t)$	$O(\log t)$	Exponentiation	Low Order
Isogeny-based	Transparent	Yes	$O(1)$	$O(1)$	Isogeny Eval.	Isogeny
Lattice-based	Transparent	Yes	$O(\log t)$	$O(\log t)$	Lattice Ops.	Lattice
AOW (this work)	**Transparent**	**Yes**	$O(\lambda \log t)$	$O(\lambda \log t)$	Field Ops. (STARK)	**AIIP**

8. Conclusion

We have introduced *Affine One-Wayness (AOW)*, a novel temporal verification primitive based on iterative polynomial evaluation over finite fields that provides:

- **Transparent setup** without trusted authorities or hidden parameters.
- **Post-quantum security** rooted in the dual hardness foundations of the Affine Iterated Inversion Problem (AIIP).
- **Efficient zero-knowledge verification** via integration with STARK proof systems.

By reducing security to the Affine Iterated Inversion Problem (AIIP) with its dual foundations in number theory (Theorem 2.1) and multivariate algebra (Theorem 2.2), AOW achieves a robust security posture resistant to advances in either cryptanalytic domain. Our security reduction (Theorem 3.2) and temporal binding proof (Theorem 3.1) establish formal guarantees under well-defined complexity assumptions (Assumption 2.1). As the reliability component of the CASH (Chaotic Affine Secure Hash) framework, AOW enables practical applications in:

- Byzantine-resistant event ordering (Section 6.1).
- Distributed synchronization (Section 6.2).
- Verifiable delay protocols with quantum resistance.

The construction's integration with STARK proof systems (Theorem 5.1) ensures verifiable computation with logarithmic scaling, making it suitable for real-world deployment in distributed systems and blockchain protocols.

Future work. We will focus on several important directions:

1. Optimizing STARK parameters for specific use cases and conducting concrete performance benchmarks.
2. Developing batch verification techniques to enhance efficiency in multi-party settings.
3. Investigating additional polynomial families beyond quadratic maps for improved security-performance tradeoffs.
4. Exploring concrete implementations for blockchain consensus mechanisms and distributed randomness generation.
5. Achieving tighter security reductions to AIIP and refining practical verification efficiency for large-scale deployment.

As the reliability component of the CASH framework, AOW provides the necessary temporal binding for the future integration of the Semantic Holder SH primitive. The SH primitive will be responsible for *legal opposability*, translating the reliable, verifiable computation traces generated by AOW into legally admissible and interpretable statements through algebraic extraction methods. The complete CASH framework, integrating CEE (confidentiality), AOW (reliability), and SH (opposability), will offer a comprehensive foundation for building transparent, secure, and legally cognizant distributed systems.

Appendix A. Implementation Notes

Appendix A.1. Field Arithmetic Optimization

For efficient implementation, we recommend using fields $\mathbb{F}_q$ with $q = 2^k - c$ for small c (Mersenne-like primes). This enables:

- Efficient modular reduction using Barrett reduction.
- Fast multiplication using shift-and-add operations.
- Optimization of squaring operations (critical for $f(x) = x^2 + \alpha$).

Appendix A.2. Polynomial Evaluation

The iterative evaluation of $f^{(n)}(x)$ can be optimized by precomputing powers of 2 modulo q. For $f(x) = x^2 + \alpha$, each step requires exactly one squaring and one addition operation.

Appendix A.3. STARK Implementation

Key optimization points for the STARK prover:

- Use radix-2 FFT for polynomial interpolation and evaluation.
- Implement FRI protocol with early termination.
- Use cryptographic hash function with efficient Merkle tree construction.

Appendix A.4. Security Considerations

1. Ensure constant-time implementation of field arithmetic to prevent timing attacks.
2. Validate polynomial $f \in \mathcal{F}_2$ (e.g., $f(x) = x^2 + \alpha$ with $\alpha \neq 0$).
3. Use cryptographically secure hash function for temporal event chain.

Appendix B. Detailed Proofs

This section content detailed proofs for theorems and propositions mentionned in the main text. Additional materials are available at [26].

Appendix B.1. Complete Proof of Theorem 3.1

Proof Appendix B.1. *We prove each property of temporal binding (Definition 2.4) comprehensively.* ***Part 1: Sequential Evaluation*** *The computation of $f^{(n)}(x)$ requires evaluating the complete sequence:*

$$x_0 = x, \quad x_1 = f(x_0), \quad x_2 = f(x_1), \quad \ldots, \quad x_n = f(x_{n-1}) = f^{(n)}(x) \tag{B.1}$$

Each evaluation $x_{i+1} = f(x_i)$ exhibits strict sequential dependency. Formally, for any parallel algorithm $\mathcal{A}$ with $m < n$ processors:

$$T_p(n) \geq \left\lceil \frac{n}{p} \right\rceil \cdot t_f \tag{B.2}$$

where t_f is time for one evaluation of f. For $p = poly(n)$, $T_p(n) = \Omega(n)$. ***Part 2: Depth Uniqueness*** *For $n \neq m \leq n_{\max}$, assume WLOG $n < m$. Then:*

$$\Pr_{x \leftarrow \mathbb{F}_q}[f^{(n)}(x) = f^{(m)}(x)] \leq \sum_{\ell | (m-n)} \frac{\ell \cdot 2^\ell}{q} \leq \frac{(m-n) \cdot 2^{m-n}}{q} \tag{B.3}$$

Since $m - n \leq n_{\max} \leq q^{1/4}$ and $q \geq 2^{2\lambda}$:

$$\frac{(m-n) \cdot 2^{m-n}}{q} \leq \frac{q^{1/4} \cdot 2^{q^{1/4}}}{2^{2\lambda}} \leq negl(\lambda) \tag{B.4}$$

Part 3: Inversion Hardness *Finding x' such that $f^{(n)}(x') = y$ is exactly the AIIP problem. By Assumption 2.1, for any PPT adversary $\mathcal{A}$:*

$$\Pr[\mathcal{A}(f, n, y) \text{ outputs } x' \text{ with } f^{(n)}(x') = y] \leq negl(\lambda) \tag{B.5}$$

Appendix B.2. Complete Proof of Theorem 3.2

Proof Appendix B.2. *We prove the security of AOW via three independent reductions to the hardness of AIIP, MQ, and HCDLP. The overall advantage of any adversary $\mathcal{A}$ against AOW is bounded by the minimum of the advantages derived from these reductions.*

Notation.. Let $\mathsf{Game}^{\mathsf{forge}}_{\mathcal{A}}(\lambda)$ be the temporal forgery game from Definition 3.2. We denote the advantage of $\mathcal{A}$ as:

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) = \Pr[\mathsf{Game}^{\mathsf{forge}}_{\mathcal{A}}(\lambda) = 1].$$

1. Reduction to AIIP (Non-Tight) *We construct a reduction algorithm $\mathcal{B}_{AIIP}$ that uses an adversary $\mathcal{A}$ to solve the AIIP problem. $\mathcal{B}_{AIIP}$ receives an AIIP challenge (f, n^*, y^*) where $y^* = f^{(n^*)}(x^*)$ for some unknown x^*.*

1. *$\mathcal{B}_{AIIP}$ runs $\mathcal{A}$ on input f.*
2. *For each query (x_i, n_i) from $\mathcal{A}$:*
   - *If $n_i = n^*$ and $f^{(n_i)}(x_i) = y^*$, $\mathcal{B}_{AIIP}$ outputs x_i as the solution to the AIIP challenge.*
   - *Otherwise, $\mathcal{B}_{AIIP}$ computes $y_i = f^{(n_i)}(x_i)$ and returns it to $\mathcal{A}$.*
3. *If $\mathcal{A}$ outputs a forgery (x^*, n^*, y^*) such that $f^{(n^*)}(x^*) = y^*$ and no query was made for this output, $\mathcal{B}_{AIIP}$ outputs x^*.*

Let E be the event that $\mathcal{A}$ wins the forgery game by forging at iteration depth n^*. The probability that $\mathcal{B}_{AIIP}$ correctly guesses n^* is $1/n_{\max}$. Thus,

$$\Pr[\mathcal{B}_{AIIP} \text{ solves AIIP}] \geq \frac{\Pr[E]}{n_{\max}} = \frac{\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda)}{n_{\max}}.$$

By Assumption 2.1, $\Pr[\mathcal{B}_{AIIP} \text{ solves AIIP}] \leq \epsilon_{AIIP}$, so:

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq n_{\max} \cdot \epsilon_{AIIP}.$$

2. Reduction to MQ (Non-Tight) *From Theorem 2.2, any instance of AIIP can be reduced to an instance of the MQ problem. We construct a reduction algorithm $\mathcal{B}_{MQ}$ that transforms an MQ challenge into an AOW instance, uses $\mathcal{A}$ to solve it, and then converts the solution back to solve the MQ challenge.*

The reduction involves unrolling the polynomial iteration into a system of quadratic equations. The loss factor comes from the need to guess the correct iteration path among $O(n_{\max})$ possibilities. A careful hybrid argument shows:

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq n_{\max} \cdot \epsilon_{MQ}.$$

3. Reduction to HCDLP (Tight) *This reduction leverages the tight equivalence established in Theorem 2.1. We construct a reduction algorithm $\mathcal{B}_{HCDLP}$ that uses an adversary $\mathcal{A}$ against AOW to solve HCDLP.*

1. *$\mathcal{B}_{HCDLP}$ receives an HCDLP instance in the Jacobian of a hyperelliptic curve C of genus $g = 2^{n-1} - 1$.*
2. *Using the constructive embedding from Theorem 2.1, $\mathcal{B}_{HCDLP}$ transforms the HCDLP instance into an AIIP instance (f, n, y), where $f(x) = x^2 + \alpha$ and α is a quadratic non-residue.*
3. *$\mathcal{B}_{HCDLP}$ runs $\mathcal{A}$ on the AOW instance defined by (f, n, y). Note that AOW is a specific instantiation of AIIP.*
4. *If $\mathcal{A}$ outputs a valid solution x such that $f^{(n)}(x) = y$, $\mathcal{B}_{HCDLP}$ uses the inverse transformation from Theorem 2.1 to extract the solution to the original HCDLP instance from x.*

This reduction is tight because the transformations in both directions are efficient and deterministic. The advantage is preserved exactly:

$$\mathsf{Adv}^{\mathsf{HCDLP}}_{\mathcal{B}_{HCDLP}} = \mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda).$$

By Assumption 2.2, $\mathsf{Adv}^{\mathsf{HCDLP}}_{\mathcal{B}_{HCDLP}} \leq \epsilon_{HCDLP}$, so:

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq \epsilon_{HCDLP}.$$

Combining the Reductions *Since the advantage of $\mathcal{A}$ is bounded by each of the above, we have:*

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq \min\left\{n_{\max} \cdot \epsilon_{AIIP}, n_{\max} \cdot \epsilon_{MQ}, \epsilon_{HCDLP}\right\}.$$

Under Assumptions 2.1, 2.3, and 2.2, each term on the right is negligible in λ for the parameters in Table 1. Therefore, $\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda)$ is negligible, proving the theorem.

The tight reduction to HCDLP provides the strongest security guarantee, as it does not suffer from any multiplicative loss.

Appendix B.3. Complete Proof of Theorem 4.1

Proof Appendix B.3. *We analyze quantum attack strategies:* ***1. Grover's Algorithm:***

$$T_{Grover} = O(n \cdot \sqrt{q}) = O(\lambda \cdot 2^{\lambda}) \quad \textit{(exponential)} \tag{B.6}$$

2. Quantum Algebraic Attacks: *Linearization requires dealing with $O(2^n)$ monomials, complexity remains $O(2^{n/2}) = O(2^{\lambda/2})$* ***3. Underlying Problems:***

- *HCDLP: $O(2^g) = O(2^{2^{n-1}})$ (doubly exponential).*
- *MQ: $O(2^{n/2}) = O(2^{\lambda/2})$ (exponential).*

4. Quantum Cycle Finding: *Requires $O(2^{\lambda/2})$ operations (exponential) All strategies require exponential time, providing λ-bit security.*

Appendix B.4. Complete Proof of Theorem 5.1

Proof Appendix B.4. ***Trace Structure:***

- *Rows:* n.
- *Columns:* 2.
- *Constraint degree:* 2.

STARK Parameters:

- *Blow-up factor:* $\rho = 4$.
- *Security parameter:* λ.
- *Queries:* $s = \lambda / \log \rho = \lambda/2$.

Complexity Analysis:

- *Proof size:* $O(\lambda \log n)$ *field elements.*
- *Verification:* $O(\lambda \log n)$ *operations.*
- *Prover:* $O(n \log n)$ *operations.*

Appendix B.5. Complete Proof of Proposition 5.1

Proof Appendix B.5. *Construct simulator* $\mathcal{S}$*:*

1. *Generate random trace* T' *satisfying:*
   - $T'[n-1, 1] = y$.
   - $T'[i, j] \leftarrow \mathbb{F}_q$ *uniform random.*
2. *Program constraints:* $\alpha = T'[i, 1] - T'[i, 0]^2$.
3. *Run STARK simulator with* (T', f, n, y).
4. *Output simulated proof.*

Indistinguishability: *For any PPT distinguisher* $\mathcal{D}$*:*

$$|\Pr[\mathcal{D}(\mathit{real}) = 1] - \Pr[\mathcal{D}(\mathcal{S}) = 1]| \leq \mathit{negl}(\lambda) \tag{B.7}$$

Appendix B.6. Complete Proof of Theorem 6.1

Proof Appendix B.6. ***Reduction to AIIP:*** *Construct* $\mathcal{B}$ *that:*

1. *Programs random oracle:* $H(e_j \| \tau_{i-1}) = x^*$.
2. *Runs* $\mathcal{A}$ *on partial chain.*
3. *Extracts* $x' = H(e_i \| \tau')$ *from forgery attempt.*

Probability:

$$\Pr[\mathcal{B} \textit{ solves AIIP}] \geq \epsilon - \mathit{negl}(\lambda) \tag{B.8}$$

Sequential Work: *Even honest recomputation requires* j *sequential steps by Theorem 3.1.*

Appendix B.7. Complete Proof of Proposition 6.1

Proof Appendix B.7. ***Security Argument:***

1. *Temporal binding (Theorem 3.1) ensures n_r sequential steps.*
2. *STARK soundness guarantees correct computation.*
3. *Cryptographic binding prevents state reuse.*

Reduction: *Any adversary producing valid proof without sequential work can be used to either:*

- *Break STARK soundness, or*
- *Violate temporal binding (contradicting Assumption 2.1).*

Appendix B.8. Complete Proof of Theorem 4.1

Proof Appendix B.8. *We analyze quantum attack strategies comprehensively:*

1. Grover's Algorithm: *Given $y = f^{(n)}(x)$, define oracle $O_y(z) = 1$ iff $f^{(n)}(z) = y$. Grover finds preimage with $O(\sqrt{q})$ queries. Each query evaluates $f^{(n)}$, requiring n sequential steps. Total quantum time:*

$$T_{Grover} = O(n \cdot \sqrt{q}) = O(\lambda \cdot 2^{\lambda}) \tag{B.9}$$

2. Quantum Algebraic Attacks: *The polynomial $f^{(n)}$ has degree $d^n = 2^n$. Quantum algorithms for polynomial equations provide at most quadratic speedup. The HHL algorithm for linear systems doesn't apply to our nonlinear system. Quantum walk on algebraic varieties requires $O(2^{n/2})$ steps, still exponential.*

3. Underlying Problems:

- ***HCDLP:*** *Genus $g = 2^{n-1} - 1$. Best quantum algorithm: $O(q^{g/2}) = O(2^{2^{n-1} \cdot \lambda})$ (doubly exponential)*
- ***MQ:*** *Grover on solution space: $O(q^{n/2}) = O(2^{\lambda})$*

4. Quantum Period Finding: *Shor's algorithm requires abelian group structure. The map $x \mapsto f^{(n)}(x)$ lacks this structure. Functional graph has varying cycle lengths, preventing quantum Fourier transform application.*

All strategies require $\Omega(2^{\lambda/2})$ quantum operations minimum.

Appendix B.9. Complete Proof of Proposition 4.1

Proof Appendix B.9. ***Degree Growth:*** *For $f(x) = x^2 + \alpha$, prove by induction that $\deg(f^{(n)}) = 2^n$:*

- *Base: $\deg(f^{(1)}) = \deg(f) = 2 = 2^1$*
- *Step: Assume $\deg(f^{(k)}) = 2^k$. Then:*

$$\deg(f^{(k+1)}) = \deg(f \circ f^{(k)}) = \deg(f) \cdot \deg(f^{(k)}) = 2 \cdot 2^k = 2^{k+1} \tag{B.10}$$

Root Finding Complexity: *Solving $f^{(n)}(x) = y$ requires finding roots of degree-2^n polynomial. Best algorithms:*

- *Berlekamp: $O(2^n \log 2^n \log q) = O(n \cdot 2^n \log q)$*
- *Cantor-Zassenhaus: $O(2^{2n} \log q)$*
- *Linearization: Requires $O(2^n)$ variables, $O(2^{3n})$ operations*

For $n = 128$: $2^{128} \approx 3.4 \times 10^{38}$ operations, requiring 10^{21} years on exascale computer.

Appendix B.10. Complete Proof of Proposition 4.2

Proof Appendix B.10. ***Functional Graph Structure:*** *A random polynomial $f : \mathbb{F}_q \to \mathbb{F}_q$ induces functional graph $G = (V, E)$ where $V = \mathbb{F}_q$ and $(x, f(x)) \in E$.*

Random Mapping Model: *For degree $d \geq 2$, polynomial f approximates random mapping. Key statistics:*

- *Expected cycle length: $\lambda_c = \sqrt{\pi q/8}$*
- *Expected tail length: $\lambda_t = \sqrt{\pi q/8}$*
- *Expected rho-length: $\lambda_\rho = \sqrt{\pi q/2}$*

Cycle Detection Complexity: *Floyd's algorithm detects cycle in $O(\lambda_c + \lambda_t) = O(\sqrt{q})$ steps. Brent's optimization: $O((1 + \sqrt{3})\sqrt{q}/2) \approx O(1.37\sqrt{q})$.*

For $q = 2^{256}$: $\sqrt{q} = 2^{128}$ evaluations required.

Appendix B.11. Complete Proof of Proposition 4.3

Proof Appendix B.11. ***Attack Strategy:*** *Compute forward chain $\{f^{(i)}(x)\}_{i=0}^{n/2}$ and backward chain $\{f^{(-j)}(y)\}_{j=0}^{n/2}$. Find collision.*

Storage Requirements: *Store $\sqrt{q}$ intermediate values, each requiring $\log_2 q$ bits:*

$$\text{Memory} = \sqrt{q} \cdot \log_2 q = 2^{128} \cdot 256 \text{ bits} = 2^{136} \text{ bits} = 2^{133} \text{ bytes} \tag{B.11}$$

This equals 10^{31} TB $= 10^{19}$ exabytes, exceeding global storage capacity.

Time-Memory Tradeoff: *Hellman's tables: $T \cdot M^2 = N^2$ where $N = q$. For $M = 2^{80}$: $T = 2^{352}$, still infeasible.*

Appendix B.12. Complete Proof of Proposition 5.1

Proof Appendix B.12. ***Simulator Construction:***

Simulator $\mathcal{S}$ on input (f, n, y):

1. *Generate random trace $T' \in \mathbb{F}_q^{n \times 2}$:*
   - *$T'[i, 0] \leftarrow \mathbb{F}_q$ uniform for $i = 0, \ldots, n-1$*
   - *$T'[i, 1] \leftarrow \mathbb{F}_q$ uniform for $i = 0, \ldots, n-2$*
   - *$T'[n-1, 1] = y$ (public output)*
2. *Add masking polynomial $M(X)$ of degree $< n$:*

$$\tilde{T}[i, j] = T'[i, j] + M(\omega^i) \tag{B.12}$$

   where ω is primitive n-th root of unity.
3. *Generate STARK proof π' for $\tilde{T}$ using STARK simulator.*
4. *Output $(\tilde{T}, \pi')$.*

Indistinguishability:

For distinguisher $\mathcal{D}$:

$$|\Pr[\mathcal{D}(\mathit{Real}(x)) = 1] - \Pr[\mathcal{D}(\mathcal{S}(y)) = 1]| \tag{B.13}$$

$$\leq |\Pr[\mathcal{D}(\tilde{T}_{\mathit{real}}) = 1] - \Pr[\mathcal{D}(\tilde{T}_{\mathit{sim}}) = 1]| \tag{B.14}$$

$$+ |\Pr[\mathcal{D}(\pi_{\mathit{real}}) = 1] - \Pr[\mathcal{D}(\pi_{\mathit{sim}}) = 1]| \tag{B.15}$$

$$\leq 0 + \mathit{negl}(\lambda) \tag{B.16}$$

The first term is 0 by statistical hiding from masking polynomial. The second term is negligible by STARK simulator security.

Appendix B.13. Complete Proof of Theorem 6.1

Proof Appendix B.13. ***Reduction to AIIP:***

Assume adversary $\mathcal{A}$ can alter event ordering. Construct AIIP solver $\mathcal{B}$:

Algorithm $\mathcal{B}$:

1. *Receive AIIP challenge (f, n^*, y^*)*
2. *Simulate temporal chain for $\mathcal{A}$:*
   - *Generate events $e_1, \ldots, e_{k-1}$ normally*
   - *Program random oracle: $H(e_k \| \tau_{k-1}) = x^*$ (unknown)*
   - *Set $\tau_k = y^*$*
3. *When $\mathcal{A}$ produces altered chain with e_j before e_k:*
   - *Extract τ' such that $f^{(k-j)}(H(e_k \| \tau')) = \tau_k = y^*$*
   - *Compute $x' = H(e_k \| \tau')$*
   - *Output x' as AIIP solution*

Success Analysis: *If $\mathcal{A}$ succeeds with probability ϵ, then $\mathcal{B}$ solves AIIP with probability $\epsilon - negl(\lambda)$ (accounting for random oracle programming).*

Sequential Work Lower Bound: *Even honest recomputation requires $\Omega(n)$ sequential evaluations of f, each taking time t_f. Total time $\Omega(n \cdot t_f)$ is inherent.*

Appendix B.14. Complete Proof of Proposition 6.1

Proof Appendix B.14. ***Security by Reduction:***

Assume adversary $\mathcal{A}$ claims round r completion without computation. Two cases:

Case 1: Invalid Proof *If π_r doesn't verify, honest participants reject. STARK soundness ensures this happens with probability $\geq 1 - 2^{-\lambda}$.*

Case 2: Valid Proof Without Computation *If $\mathcal{A}$ produces valid (τ_r, π_r) without computing $f^{(n_r)}$:*

1. *Extract witness from STARK proof (knowledge soundness)*
2. *Witness contains x such that $f^{(n_r)}(x) = \tau_r$*
3. *This solves AIIP, contradicting Assumption 2.1*

Temporal Guarantee: *By Theorem 3.1, computing τ_r requires:*

$$T \geq n_r \cdot t_f = n_r \cdot \Omega(1) = \Omega(n_r) \tag{B.17}$$

No parallelization or precomputation can reduce this bound.

Appendix B.15. Complete Proof of Theorem 3.1

Proof Appendix B.15. *We prove each property of temporal binding (Definition 2.4) comprehensively.*

Part 1: Sequential Evaluation

The computation of $f^{(n)}(x)$ requires evaluating the complete sequence:

$$x_0 = x, \quad x_1 = f(x_0), \quad x_2 = f(x_1), \quad \ldots, \quad x_n = f(x_{n-1}) = f^{(n)}(x) \tag{B.18}$$

Each evaluation $x_{i+1} = f(x_i)$ exhibits strict sequential dependency. Formally, for any parallel algorithm $\mathcal{A}$ with $m < n$ processors:

$$T_p(n) \geq \left\lceil \frac{n}{p} \right\rceil \cdot t_f \tag{B.19}$$

where t_f is time for one evaluation of f. For $p = poly(n)$, $T_p(n) = \Omega(n)$.

Part 2: Depth Uniqueness

For $n \neq m \leq n_{\max}$, assume WLOG $n < m$. Then:

$$\Pr_{x \leftarrow \mathbb{F}_q}[f^{(n)}(x) = f^{(m)}(x)] \leq \sum_{\ell | (m-n)} \frac{\ell \cdot 2^\ell}{q} \leq \frac{(m-n) \cdot 2^{m-n}}{q} \tag{B.20}$$

Since $m - n \leq n_{\max} \leq q^{1/4}$ and $q \geq 2^{2\lambda}$:

$$\frac{(m-n) \cdot 2^{m-n}}{q} \leq \frac{q^{1/4} \cdot 2^{q^{1/4}}}{2^{2\lambda}} \leq negl(\lambda) \tag{B.21}$$

Part 3: Inversion Hardness

Finding x' such that $f^{(n)}(x') = y$ is exactly the AIIP problem. By Assumption 2.1, for any PPT adversary $\mathcal{A}$:

$$\Pr[\mathcal{A}(f, n, y) \text{ outputs } x' \text{ with } f^{(n)}(x') = y] \leq negl(\lambda) \tag{B.22}$$

Appendix B.16. Complete Proof of Theorem 3.2

Proof Appendix B.16. *We prove the security of AOW via three independent reductions to the hardness of AIIP, MQ, and HCDLP. The overall advantage of any adversary $\mathcal{A}$ against AOW is bounded by the minimum of the advantages derived from these reductions.*

Notation.. Let $\mathsf{Game}_{\mathcal{A}}^{\mathsf{forge}}(\lambda)$ be the temporal forgery game from Definition 3.2. We denote the advantage of $\mathcal{A}$ as:

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{AOW}}(\lambda) = \Pr[\mathsf{Game}_{\mathcal{A}}^{\mathsf{forge}}(\lambda) = 1].$$

1. Reduction to AIIP (Non-Tight) *We construct a reduction algorithm $\mathcal{B}_{AIIP}$ that uses an adversary $\mathcal{A}$ to solve the AIIP problem. $\mathcal{B}_{AIIP}$ receives an AIIP challenge (f, n^*, y^*) where $y^* = f^{(n^*)}(x^*)$ for some unknown x^*.*

1. *$\mathcal{B}_{AIIP}$ runs $\mathcal{A}$ on input f.*
2. *For each query (x_i, n_i) from $\mathcal{A}$:*
   - *If $n_i = n^*$ and $f^{(n_i)}(x_i) = y^*$, $\mathcal{B}_{AIIP}$ outputs x_i as the solution to the AIIP challenge.*
   - *Otherwise, $\mathcal{B}_{AIIP}$ computes $y_i = f^{(n_i)}(x_i)$ and returns it to $\mathcal{A}$.*

3. *If $\mathcal{A}$ outputs a forgery (x^*, n^*, y^*) such that $f^{(n^*)}(x^*) = y^*$ and no query was made for this output, $\mathcal{B}_{AIIP}$ outputs x^*.*

Let E be the event that $\mathcal{A}$ wins the forgery game by forging at iteration depth n^*. The probability that $\mathcal{B}_{AIIP}$ guesses the correct n^* is $1/n_{\max}$. Thus,

$$\Pr[\mathcal{B}_{AIIP} \text{ solves } AIIP] \geq \frac{\Pr[E]}{n_{\max}} = \frac{\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda)}{n_{\max}}.$$

By Assumption 2.1, $\Pr[\mathcal{B}_{AIIP} \text{ solves } AIIP] \leq \epsilon_{AIIP}$, so:

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq n_{\max} \cdot \epsilon_{AIIP}.$$

2. Reduction to MQ (Tighter) *We construct a reduction algorithm $\mathcal{B}_{MQ}$ that reduces solving AIIP to solving a system of multivariate quadratic equations. From Theorem 2.2, the AIIP problem for iteration depth n can be reduced to solving an MQ system with $O(n)$ equations and variables.*

The reduction uses a hybrid argument over $\sqrt{n_{\max}}$ checkpoints. Specifically, $\mathcal{B}_{MQ}$ guesses an intermediate iteration j uniformly from $[0, \sqrt{n_{\max}}]$. The probability of guessing correctly is $1/\sqrt{n_{\max}}$. The MQ system is constructed to encode the computation from iteration j to n^*, reducing the loss factor to $\sqrt{n_{\max}}$.

Thus,

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq \sqrt{n_{\max}} \cdot \epsilon_{MQ}.$$

3. Reduction to HCDLP (Tight) *We construct a reduction algorithm $\mathcal{B}_{HCDLP}$ that uses $\mathcal{A}$ to solve the discrete logarithm problem in the Jacobian of a high-genus hyperelliptic curve. From Theorem 2.1, solving AIIP for $f(x) = x^2 + \alpha$ (with α a quadratic non-residue) and iteration depth n is equivalent to solving the DLP in the Jacobian of a hyperelliptic curve of genus $g_n = 2^{n-1} - 1$.*

The reduction is direct: given an HCDLP instance, $\mathcal{B}_{HCDLP}$ embeds it into an AIIP instance using the transformation from Theorem 2.1. Then, $\mathcal{B}_{HCDLP}$ runs $\mathcal{A}$ on this AIIP instance. If $\mathcal{A}$ outputs a forgery, $\mathcal{B}_{HCDLP}$ extracts the solution to the HCDLP instance.

Crucially, this reduction is tight*: there is no multiplicative loss in the advantage. Specifically,*

$$\mathsf{Adv}^{\mathsf{HCDLP}}_{\mathcal{B}_{HCDLP}} = \mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda).$$

By Assumption 2.2, $\mathsf{Adv}^{\mathsf{HCDLP}}_{\mathcal{B}_{HCDLP}} \leq \epsilon_{HCDLP}$, so:

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq \epsilon_{HCDLP}.$$

Combining the Reductions *Since the advantage of $\mathcal{A}$ is bounded by each of the above, we have:*

$$\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda) \leq \min\left\{n_{\max} \cdot \epsilon_{AIIP}, \sqrt{n_{\max}} \cdot \epsilon_{MQ}, \epsilon_{HCDLP}\right\}.$$

Under Assumptions 2.1, 2.3, and 2.2, each of ϵ_{AIIP}, ϵ_{MQ}, and ϵ_{HCDLP} is negligible in λ for the parameters in Table 1. Therefore, $\mathsf{Adv}^{\mathsf{AOW}}_{\mathcal{A}}(\lambda)$ is negligible, proving the theorem.

The tight reduction to HCDLP ensures that the security of AOW is fundamentally based on a well-studied hard problem without multiplicative loss, providing strong concrete security guarantees.

Appendix B.17. Complete Proof of Theorem 5.1

Proof Appendix B.17. ***Trace Structure:***

- *Rows:* n.
- *Columns:* $d + 1$ *for degree-d polynomial.*
- *Constraint degree:* d.

STARK Parameters:

- *Blow-up factor:* $\rho = 4$.
- *Security parameter:* λ.
- *Queries:* $s = \lambda / \log \rho = \lambda/2$.

Complexity Analysis:
1. ***Proof Size:*** *The proof consists of:*

- s *Merkle authentication paths of depth* $\log(n \cdot \rho) = \log(4n)$
- *FRI commitments:* $\log n$ *rounds with* s *queries each*
- *Total:* $O(s \cdot \log n) = O(\lambda \log n)$ *field elements*

2. ***Verification Time:***

- *Verify* s *Merkle paths:* $O(s \cdot \log n)$ *hash operations*
- *Check FRI queries:* $O(s \cdot \log n)$ *field operations*
- *Total:* $O(\lambda \log n)$ *operations*

3. ***Prover Time:***

- *Compute trace:* $O(n)$ *polynomial evaluations*
- *FFT for polynomial commitment:* $O(n \log n)$
- *Merkle tree construction:* $O(n)$
- *Total:* $O(n \log n)$ *operations*

Appendix C. UC Security of AOW

Theorem Appendix C.1 (UC Realization of $\mathcal{F}_{\mathbf{AOW}}$). *The AOW protocol* Π_{AOW} *UC-realizes the ideal functionality* $\mathcal{F}_{AOW}$ *(Functionality 3.1) in the random oracle model under Assumption 2.1.*

Proof Appendix C.1. *We construct a simulator* $\mathcal{S}$ *for the ideal world:*
Simulator $\mathcal{S}$***:***

1. *Maintain table* T *of simulated evaluations*
2. *Upon adversary's query* (x, n) *to* Π_{AOW}*:*
    - *If* $(x, n, y) \in T$*, return* y

- *Else, sample* $y \leftarrow \mathbb{F}_q$*, store* (x, n, y) *in* T*, return* y

3. *Upon environment's instruction to corrupt party* P*:*

   - *Obtain real* (x, n, y) *from* $\mathcal{F}_{AOW}$
   - *Program random oracle to ensure consistency*

Indistinguishability: *The simulation is perfect unless adversary finds collision or inverts AOW, which happens with probability* $\leq n_{\max} \cdot \epsilon_{AIIP}$ *by Theorem 3.2.*

References

[1] Albrecht, M., et al. (2016). MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity. In ASIACRYPT.

[2] Basso, A., Codogni, G., Connolly, D., De Feo, L., Fouotsa, T. B., Lido, G., ... and Wesolowski, B. (2023). Supersingular curves you can trust. In Advances in Cryptology – EUROCRYPT 2023 (pp. 405–437). Springer, Cham.

[3] Bardet, M., Faugère, J. C., and Salvy, B. (2014).On the complexity of the F5 Gröbner basis algorithm. Journal of Symbolic Computation, 70, 49–70.

[4]

[5] Boneh, D., et al. (2018). Verifiable Delay Functions. In CRYPTO.

[6] Biasse, J. F., and Jacobson Jr, M. J. (2019). Practical improvements to class group computation in imaginary quadratic orders. Journal of Cryptology, 32(1), 1–31.

[7] Caminata, A., and Gorla, E. (2020). Solving multivariate polynomial systems and an invariant from commutative algebra. arXiv preprint arXiv:2003.12952.

[8] Childs, A. M., Jao, D., and Soukharev, V. (2014). Constructing elliptic curve isogenies in quantum subexponential time. Journal of Mathematical Cryptology, 8(1), 1–29.

[9] De Feo, L., Masson, S., Petit, C., and Sanso, A. (2019). Verifiable Delay Functions from Supersingular Isogenies and Pairings. In Advances in Cryptology – ASIACRYPT 2019 (pp. 248–277). Springer, Cham.

[10] Wesolowski, B. (2019). Efficient Verifiable Delay Functions. In EUROCRYPT.

[11] Pietrzak, K. (2018). Simple Verifiable Delay Functions. In ITCS.

[12] Habutsu, T., et al. (1991). A Secret Key Cryptosystem by Iterating a Chaotic Map. In EUROCRYPT.

[13] Chen, L., et al. (2016). Report on Post-Quantum Cryptography. NIST IR 8105.

[14] Ben-Sasson, E., et al. (2019). STARKs with Prover Complexity. In EUROCRYPT.

[15] Feo, L., et al. (2019). Vector Verifiable Delay Functions. In Journal of Cryptology.

[16] De Feo, L. and Masson, S. (2019). Verifiable Delay Functions from Isogenies and Pairings. In: Galbraith S., Moriai S. (eds) Advances in Cryptology – ASIACRYPT 2019. Lecture Notes in Computer Science, vol 11921. Springer, Cham.

[17] Doerner, J., et al. (2022). Reinforced Concrete: A Fast Hash Function for Verifiable Computation. In EUROCRYPT.

[18] Feo, L., et al. (2019). Vector Verifiable Delay Functions. In Journal of Cryptology.

[19] Faugère, J. C. (1999). A new efficient algorithm for computing Gröbner bases without reduction to zero (F4). Proceedings of the 1999 international symposium on Symbolic and algebraic computation (pp. 75–83).

[20] Faugère, J. C. (2002). A new efficient algorithm for computing Gröbner bases (F5). In Proceedings of the 2002 international symposium on Symbolic and algebraic computation (pp. 75–83).

[21] Schofnegger, M., et al. (2022). Arion: Arithmetization-Oriented Permutation and Hashing from GTDS. In: Dodis Y., Shrimpton T. (eds) Advances in Cryptology – CRYPTO 2022. Lecture Notes in Computer Science, vol 13507. Springer, Cham.

[22] Minka Mi Nguidjoi, T. E. (2025). The AIIP Problem: Toward a Post-Quantum Hardness Assumption from Affine Iterated Inversion over Finite Fields. In Cryptology ePrint Archive 2025/1590

[23] Minka Mi Nguidjoi, T. E. (2025). The Chaotic Entropic Expansion (CEE): A Transparent Post-Quantum Data Confidentiality Primitive via Entropic Chaotic Maps. In Cryptology ePrint Archive 2025/1615

[24] Minka Mi Nguidjoi, T. E., Mani Onana F. S., Djotio Ndié, T. (2025). The CRO Trilemma : a formal incompatibility between Confidentiality, Reliability and legal Opposability in Post-Quantum proof systems. In Cryptology ePrint Archive 2025/1348

[25] Minka Mi Nguidjoi, T. E., Mani Onana, F. S., Djotio Ndié, T., and Bouetou Bouetou, T. (2025). Quantum composable and contextual security infrastructure (Q2CSI): A modular architecture for legally explainable cryptographic signatures. Cryptology ePrint Archive, Report 2025/1380.

[26] Anonymized Directory (KryptoResearcher): The Affine One-Wayness (AOW) Implementation. GitHub repository. `https://github.com/KryptoResearcher/AOW`